

1. спираме за почивка. Може ли бележка за утре

2. ChatGPT каза:

Разбира се. Ето актуалния checkpoint за утре. Можеш просто да ми го изпратиш и ще продължим **точно оттам, без да започваме отначало**.

Писане

Checkpoint — Student Management System

1. Текуща цел

Продължаваме V1 на **Student Management System**.

Основният принцип остава:

- малки стъпки;
- една логична промяна;
- Build;
- проверка;
- следваща стъпка;
- не гадаем съществуващия код — първо го виждаме.

GitHub:

<https://github.com/esp-nv/StudentManagement/tree/main/StudentManagement>

2. Архитектура

StudentsController



IStudentService



StudentService



ApplicationDbContext



EF Core



Database

Използваме Service Pattern.

3. Какво е направено

Service / Architecture

- [✓] IStudentService

- [✓] StudentService
- [✓] Controller-ът не използва директно DbContext
- [✓] CRUD операциите са в Service
- [✓] Build успешно
- [✓] Errors: 0
- [✓] Warnings: 0

CreateStudentViewModel

Има:

- FirstName
- LastName
- Email
- Age
- Course

Текущата validation логика е:

```
[Required]
public string FirstName { get; set; } = string.Empty;

[Required]
public string LastName { get; set; } = string.Empty;

[Required]
[EmailAddress]
[StringLength(100)]
public string Email { get; set; } = string.Empty;

[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
public int Age { get; set; }

public string Course { get; set; } = string.Empty;
```

4. Age — завършена текуща реализация

Решихме **да не използваме custom validation message**.

Премахнахме:

```
ErrorMessage = ValidationMessages.AgeMustBeBetweenMinAndMax
```

и в крайна сметка ValidationMessages вече не съдържа тази Age константа.

Премахнахме и:

```
using StudentManagement.Common.Validation;
```

ОТ CreateStudentViewModel.

Текущото Age validation е:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
```

```
public int Age { get; set; }
```

5. StudentConstants

Имаме:

```
StudentConstants.MinAge  
StudentConstants.MaxAge
```

КОИТО В МОМЕНТА СА:

```
18  
100
```

Важно: **18 и 100** не ги пишем повторно, когато можем да използваме КОНСТАНТИТЕ.

6. Age в Create.cshtml

Добавихме:

```
@using StudentManagement.Common.Constants
```

и Age input-ът вече е:

```
<input asp-for="Age"  
       type="number"  
       min="@StudentConstants.MinAge"  
       max="@StudentConstants.MaxAge"  
       class="form-control" />
```

Това генерира HTML:

```
<input type="number" min="18" max="100">
```

7. Age informational hint

Добавихме:

```
<div class="age-hint">  
    Age → must be between @StudentConstants.MinAge and  
    @StudentConstants.MaxAge.  
</div>
```

Така диапазонът идва от StudentConstants.

Не пишем:

```
18  
100
```

директно в текста.

8. Age hint — без JavaScript

Искаме hint-ът да се показва само когато Age полето е във focus.

В `wwwroot/css/site.css` добавихме:

```
.age-hint {  
    display: none;  
}  
  
input[name="Age"]:focus ~ .age-hint {  
    display: block;
```

```
}
```

Първоначално използвахме +, но hint-ът не се показваше.

Причината беше, че Razor генерира допълнителен:

```
<input name="__Invariant" type="hidden" value="Age">
```

между Age input-а и .age-hint.

Затова:

```
+
```

не работеше.

Сменихме го с:

```
~
```

което означава следващ sibling елемент, дори да има друг елемент между тях.

Това вече работи. 

9. Генерираният HTML за Age

Проверихме го през Browser DevTools и получихме:

```
<input type="number"
  min="18"
  max="100"
  class="form-control"
  data-val="true"
  data-val-range="The field Age must be between 18 and 100."
  data-val-range-max="100"
  data-val-range-min="18"
  data-val-required="The Age field is required."
  id="Age"
  name="Age"
  value="0">
<input name="__Invariant" type="hidden" value="Age">

<div class="age-hint">
  Age → must be between 18 and 100.
</div>
```

10. Важното за validation съобщенията

Първоначално мислехме, че стандартното [Range] съобщение се показва под полето.

После установихме, че видимите съобщения:

„Стойността трябва да е по-голяма или равна на 18.“

и

„Стойността трябва да е по-малка или равна на 100.“

са **browser native validation popup**, породен от:

```
min="18"
max="100"
```

Тоест това НЕ е ASP.NET Core validation message в:

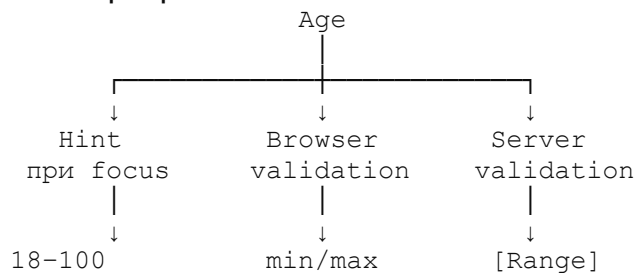
```
<span asp-validation-for="Age">
```

Това е HTML5/browser validation.

И решихме да го запазим — съобщенията са ясни и полезни.

11. Какво постигнахме с Age

Получихме три различни нива:



Hint

При focus:

Age → must be between 18 and 100.

Browser validation

Например при 17:

Стойността трябва да е по-голяма или равна на 18.

При 101:

Стойността трябва да е по-малка или равна на 100.

Server-side validation

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
```

Това остава независимо от HTML/browser validation.

12. Security / Validation принцип

Важно е да помним:

HTML min/max

↓

UX / client-side validation

[Range]

↓

server-side validation

Database constraints

↓

database-level protection

HTML ограниченията **не са security boundary**.

Ако някой промени HTML през DevTools, server-side validation трябва да остане защита.

13. Последните Build резултати

Всички последни промени са минали успешно.

Последният Build:

Build: 1 succeeded

Failed: 0

Errors: 0

Warnings: 0

14. Текущо състояние

Аге считаме за завършено на този етап. ✓

Не правим повече промени по него, освен ако по време на общата проверка не открием реален проблем.

15. Следваща задача — FirstName

Тук спираме за днес.

Утре **не започваме отначало**.

Първата следваща задача е:

FirstName

↓

какви validation правила са необходими?

↓

какви HTML ограничения имат смисъл?

↓

трябва ли да има informational hint?

↓

имаме ли нужда от нова константа?

Но не измисляме ограниченията.

Преди промяна трябва да видим актуалния:

Models/Student.cs

и ако има отделно задание/изисквания за полетата — да ги използваме.

16. Правило за утре

Продължаваме както досега:

1 малка промяна

↓

обяснение

↓

потребителят я прави

↓

Build

↓

проверка

↓

следваща стъпка

Не променяме няколко файла наведнъж, освен ако промяната логично не го изисква.

При JavaScript остава правилото:

Не използваме JS, ако HTML + CSS + Razor могат да свършат работата.

За Age успяхме без JS. ✓

Точна начална точка за утре

Първо:

Student.cs

↓

FirstName

↓

определяме точните validation изисквания

След това една малка промяна → Build → проверка.

Не започваме проекта отначало. Продължаваме от FirstName.

Почини си спокойно. 😊 Утре продължаваме директно от **FirstName**.

1. Ще ти давам **промените стъпка по стъпка**, като за всяка промяна ще посочвам:
 1. **Кой файл** отваряш.
 2. **Къде точно** отиваш.
 3. **Какво да намериш**.
 4. **Какво да замениш/добавиш**.
 5. Кодът в готов вид.
 6. **Коментар под кода** — ще обяснявам какво прави промяната и защо я правим.
 7. Ти правиш промяната.
 8. Правим **Build** и проверяваме резултата.
 9. Едва тогава преминаваме към следващата стъпка.

И няма да ти давам 5 промени наведнъж. **Една промяна → Build → проверка.** ✓

Стъпка 1 — покажи ми Student.cs

Отвори:

StudentManagement

└─ Models

└─ Student.cs

Копирай **целия текущ код** на Student.cs и го изпрати тук.

След това ще ти кажа точно **първата промяна за** FirstName.

2. Вие казахте:

```
using System.ComponentModel.DataAnnotations; namespace
StudentManagement.Models { public class Student { public int Id {
get; set; } [Required] public string FirstName { get; set; } =
string.Empty; [Required] public string LastName { get; set; } =
string.Empty; [Required] [EmailAddress] public string Email { get; set; }
= string.Empty; public string Course { get; set; } = string.Empty; public
int Age { get; set; } } }
```

3. ChatGPT каза:

Да. Виждаме текущия Student.cs. 👍

За **FirstName** в момента имаме само:

```
[Required]
public string FirstName { get; set; } = string.Empty;
```

Но преди да добавяме каквото и да е, ще направим **само една малка промяна**.

Стъпка 1 — **FirstName**: максимална дължина

Първо ще добавим [StringLength(100)], по същия принцип, който вече използваме за Email.

Файл

Отвори:

Models/Student.cs

Намери:

```
[Required]
public string FirstName { get; set; } = string.Empty;
```

и го **замени с**:

```
[Required]
[StringLength(100)]
public string FirstName { get; set; } = string.Empty;
```

Коментар

[Required] означава, че FirstName не може да бъде празен.

[StringLength(100)] поставя максимална дължина от 100 символа.

Използваме 100 тук, защото това вече е установеното ограничение в проекта за текстовото поле Email; но **следващата стъпка ще бъде да проверим дали моделът/изискванията имат конкретно правило за FirstName**, преди да добавяме други ограничения.

Сега **не променяй нищо друго**.

Направи промяната и пусни **Build**. Кажи ми резултата от Build-а и продължаваме.

4. Вие казахте:

магическите числа ги изнасяме в файл

5. ChatGPT каза:

Точно така. 👍 Добре, че го хвана.

В проекта сме приели правилото: **магическите числа не ги пишем директно в кода**, а ги изнасяме в StudentConstants.

Затова **не добавяй** [StringLength(100)]. Връщаме се една стъпка и първо проверяваме какво вече имаме.

Стъпка 1 — проверяваме StudentConstants

Отвори:

```
StudentManagement
├── Common
│   └── Constants
│       └── StudentConstants.cs
```

Изпрати ми **целия текущ код** на StudentConstants.cs.

После ще определим дали имаме подходяща константа за FirstName и ще направим само необходимата промяна.

Правило за проекта:

18, 100 и други стойности, които представляват правила/ограничения, **не ги разпръскваме из кода**. Използваме именувани константи.

6. Вие казахте:

```
namespace StudentManagement.Common.Constants; public static
class StudentConstants { public const int MinAge = 18; public const
int MaxAge = 100; }
```

7. ChatGPT каза:

Точно. В момента StudentConstants съдържа само ограниченията за Age:

```
public const int MinAge = 18;
public const int MaxAge = 100;
```

Затова сега правим **само една промяна** — добавяме константа за максималната дължина на името.

Стъпка 1 — добавяме MaxNameLength

Файл

Отвори:

```
Common/Constants/StudentConstants.cs
```

Намери:

```
public const int MinAge = 18;
```

```
public const int MaxAge = 100;
```

и го промени на:

```
public const int MinAge = 18;  
public const int MaxAge = 100;  
public const int MaxNameLength = 100;
```

Коментар

MaxNameLength е именувана константа за максималната дължина на име.

Така вместо да пишем магическото число 100 директно във validation атрибута, по-късно ще използваме:

```
StudentConstants.MaxNameLength
```

Това прави правилото централизирано и по-лесно за промяна.

Сега спираме дотук. Не променяй още Student.cs.

Направи тази промяна и пусни **Build**. Кажи ми резултата.

1. Build started at 8:48 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 8:48 PM and took 06.170 seconds =====

2. ChatGPT каза:

Отлично. 

Build:

- o 1 succeeded
- o 0 failed
- o 0 errors
- o 0 warnings

Константата е добавена успешно.

Стъпка 2 — използваме константата за `FirstName`

Сега вече можем да променим Student.cs.

Файл

Отвори:

```
Models/Student.cs
```

Най-горе, след:

```
using System.ComponentModel.DataAnnotations;
```

добави:

```
using StudentManagement.Common.Constants;
```

Така началото на файла ще стане:

```
using System.ComponentModel.DataAnnotations;  
using StudentManagement.Common.Constants;
```

```
namespace StudentManagement.Models
```

Коментар

Добавяме using, за да може Student.cs да използва:
StudentConstants.MaxLength

без да изписваме пълния namespace всеки път.

Само тази промяна засега. Не добавяй още [StringLength].

Направи промяната и пусни **Build**. След резултата продължаваме.
ВлизанеРегистрирайте се безплатно

1. Вие казахте:

може ли да продължим с проекта

2. ChatGPT каза:

Разбира се! 👍 Нека продължим.

Кажи ми само **кой проект** имаш предвид и откъде да подхванем — можеш да ми пуснеш последния вариант, кода/текста или просто да ми напомниш с 1–2 изречения какво правехме.

3. Вие казахте:

спиране за почивка. Може ли бележка за утре 2. ChatGPT каза: Разбира се. Ето актуалния checkpoint за утре. Можеш просто да ми го изпратиш и ще продължим точно оттам, без да започваме отначало. Писане ✨ Checkpoint — Student Management System 1. Текуща цел Продължаваме V1 на Student Management System. Основният принцип остава: о малки стъпки; о една логична промяна; о Build; о проверка; о следваща стъпка; о не гадаем съществуващия код — първо го виждаме. GitHub: <https://github.com/espnv/StudentManagement/tree/main/StudentManagement> 2. Архитектура StudentsController ↓ IStudentService ↓ StudentService ↓ ApplicationDbContext ↓ EF Core ↓ Database Използваме Service Pattern. 3. Какво е направено Service / Architecture о [✓] IStudentService о [✓] StudentService о [✓] Controller-ът не използва директно DbContext о [✓] CRUD операциите са в Service о [✓] Build успешно о [✓] Errors: 0 о [✓] Warnings: 0 CreateStudentViewModel Има: о FirstName о LastName о Email о Age о Course Текущата validation логика е: [Required] public string FirstName { get; set; } =

string.Empty; [Required] public string LastName { get; set; } =
 string.Empty; [Required] [EmailAddress] [StringLength(100)] public
 string Email { get; set; } = string.Empty;
 [Range(StudentConstants.MinAge, StudentConstants.MaxAge)] public
 int Age { get; set; } public string Course { get; set; } = string.Empty; 4.
 Age — завършена текуща реализация Решихме да не използваме
 custom validation message. Премахнахме: ErrorMessage =
 ValidationMessages.AgeMustBeBetweenMinAndMax и в крайна
 сметка ValidationMessages вече не съдържа тази Age константа.
 Премахнахме и: using StudentManagement.Common.Validation; от
 CreateStudentViewModel. Текущото Age validation е:
 [Range(StudentConstants.MinAge, StudentConstants.MaxAge)] public
 int Age { get; set; } 5. StudentConstants Имаме:
 StudentConstants.MinAge StudentConstants.MaxAge които в мо-
 мента са: 18 100 Важно: 18 и 100 не ги пишем повторно, когато
 можем да използваме константите. 6. Age в Create.cshtml Доба-
 вихме: @using StudentManagement.Common.Constants и Age
 input-ът вече е: <input asp-for="Age" type="number"
 min="@StudentConstants.MinAge"
 max="@StudentConstants.MaxAge" class="form-control" /> Това ге-
 нерира HTML: <input type="number" min="18" max="100"> 7. Age
 informational hint Добавихме: <div class="age-hint"> Age → must
 be between @StudentConstants.MinAge and
 @StudentConstants.MaxAge. </div> Така диапазонът идва от
 StudentConstants. Не пишем: 18 100 директно в текста. 8. Age hint
 — без JavaScript Искаме hint-ът да се показва само когато Age по-
 лето е във focus. В wwwroot/css/site.css добавихме: .age-hint {
 display: none; } input[name="Age"]:focus ~ .age-hint { display: block; }
 Първоначално използвахме +, но hint-ът не се показваше. Причи-
 ната беше, че Razor генерира допълнителен: <input
 name="__Invariant" type="hidden" value="Age"> между Age input-а
 и .age-hint. Затова: + не работеше. Сменихме го с: ~ което озна-
 чава следващ sibling елемент, дори да има друг елемент между
 тях. Това вече работи. ✓ 9. Генерираният HTML за Age Прове-
 рихме го през Browser DevTools и получихме: <input
 type="number" min="18" max="100" class="form-control" data-
 val="true" data-val-range="The field Age must be between 18 and
 100." data-val-range-max="100" data-val-range-min="18" data-val-

required="The Age field is required." id="Age" name="Age" value="0"> <input name="__Invariant" type="hidden" value="Age"> <div class="age-hint"> Age → must be between 18 and 100. </div>

10. Важното за validation съобщенията Първоначално мислехме, че стандартното [Range] съобщение се показва под полето. После установихме, че видимите съобщения: „Стойността трябва да е по-голяма или равна на 18.“ и „Стойността трябва да е по-малка или равна на 100.“ са browser native validation popup, породен от: min="18" max="100" Тоест това НЕ е ASP.NET Core validation message в: Това е HTML5/browser validation. И решихме да го запазим — съобщенията са ясни и полезни.

11. Какво постигнахме с Age Получихме три различни нива:

Age		_____		↓↓↓	Hint Browser
Server при focus validation				↓↓↓	18–100 min/max [Range] Hint

При focus: Age → must be between 18 and 100. Browser validation Например при 17: Стойността трябва да е по-голяма или равна на 18. При 101: Стойността трябва да е по-малка или равна на 100. Server-side validation [Range(StudentConstants.MinAge, StudentConstants.MaxAge)] Това остава независимо от HTML/browser validation.

12. Security / Validation принцип Важно е да помним: HTML min/max ↓ UX / client-side validation [Range] ↓ server-side validation Database constraints ↓ database-level protection HTML ограниченията не са security boundary. Ако някой промени HTML през DevTools, server-side validation трябва да остане защита.

13. Последните Build резултати Всички последни промени са минали успешно. Последният Build: Build: 1 succeeded Failed: 0 Errors: 0 Warnings: 0

14. Текущо състояние Age считаме за завършено на този етап. ☒ Не правим повече промени по него, освен ако по време на общата проверка не открием реален проблем.

15. Следваща задача — FirstName Тук спираме за днес. Утре не започваме отначало. Първата следваща задача е: FirstName ↓ какви validation правила са необходими? ↓ какви HTML ограничения имат смисъл? ↓ трябва ли да има informational hint? ↓ имаме ли нужда от нова константа? Но не измисляме ограниченията. Преди промяна трябва да видим актуалния: Models/Student.cs и ако има отделно задание/изисквания за полетата — да ги използваме.

16. Правило за утре Продължаваме както досега: 1 малка промяна ↓ обяснение ↓ потребителят я прави ↓ Build ↓ проверка ↓

следваща стъпка Не променяме няколко файла наведнъж, освен ако промяната логично не го изисква. При JavaScript остава правилото: Не използваме JS, ако HTML + CSS + Razor могат да свършат работата. За Age успяхме без JS. ✅ 🔄 Точна начална точка за утре Първо: Student.cs ↓ FirstName ↓ определяме точните validation изисквания След това една малка промяна → Build → проверка. Не започваме проекта отначало. Продължаваме от FirstName. Почини си спокойно. 😊 Утре продължаваме директно от FirstName.

4. ChatGPT каза:

Разбира се. Запазваме това като **checkpoint за утре** в рамките на този разговор.

📌 **Утре продължаваме оттук**

Проект: Student Management System

Текущ етап: V1 — validation

Последно завършено: Age ✅

Build: 1 succeeded, 0 failed, 0 errors, 0 warnings

Точната начална точка утре е:

Models/Student.cs → FirstName

Ще:

1. видим актуалния Student.cs;
2. определим реалните изисквания за FirstName;
3. решим validation правилата;
4. преценим HTML ограниченията и евентуален hint;
5. направим **само една малка промяна**;
6. Build → проверка → следваща стъпка.

Важно: няма да гадаем и няма да започваме проекта отначало.

Ще работим върху реалния код, който имаш.

Почивка заслужена. 😊 Утре директно от **FirstName**.

5. Вие казахте:

да продължим

6. ChatGPT каза:

Да, продължаваме директно от checkpoint-а. 👍

Следващата ни стъпка е **FirstName**, но според уговорката първо гледаме реалния код, без да измисляме ограничения.

Моля, отвори:

Models/Student.cs

и ми изпрати **целия текущ файл** тук.

После ще го разгледаме заедно и ще направим само **първата малка промяна**, след което ще направим Build.

7. Вие казахте:

но промените ми даваш стъпка по стъпка кое и къде да променя, коментари под кода

8. ChatGPT каза:

Да — точно така ще работим. 👍

Ще ти давам **промените стъпка по стъпка**, като за всяка промяна ще посочвам:

1. **Кой файл** отваряш.
2. **Къде точно** отиваш.
3. **Какво да намериш**.
4. **Какво да замениш/добавиш**.
5. Кодът в готов вид.
6. **Коментар под кода** — ще обяснявам какво прави промяната и защо я правим.
7. Ти правиш промяната.
8. Правим **Build** и проверяваме резултата.
9. Едва тогава преминаваме към следващата стъпка.

И няма да ти давам 5 промени наведнъж. **Една промяна → Build → проверка.** ✅

Стъпка 1 — покажи ми Student.cs

Отвори:

StudentManagement

└─ Models

 └─ Student.cs

Копирай **целия текущ код** на Student.cs и го изпрати тук.

След това ще ти кажа точно **първата промяна за** FirstName.

9. Вие казахте:

```
using System.ComponentModel.DataAnnotations; namespace
StudentManagement.Models { public class Student { public int Id {
get; set; } [Required] public string FirstName { get; set; } =
string.Empty; [Required] public string LastName { get; set; } =
string.Empty; [Required] [EmailAddress] public string Email { get; set; }
= string.Empty; public string Course { get; set; } = string.Empty; public
int Age { get; set; } } }
```

10. ChatGPT каза:

Да. Виждаме текущия Student.cs. 👍

За **FirstName** в момента имаме само:

```
[Required]
public string FirstName { get; set; } = string.Empty;
```

Но преди да добавяме каквото и да е, ще направим **само една малка промяна**.

Стъпка 1 — **FirstName**: максимална дължина

Първо ще добавим [StringLength(100)], по същия принцип, който вече използваме за Email.

Файл

Отвори:

Models/Student.cs

Намери:

```
[Required]
public string FirstName { get; set; } = string.Empty;
```

и го **замени с**:

```
[Required]
[StringLength(100)]
public string FirstName { get; set; } = string.Empty;
```

Коментар

[Required] означава, че FirstName не може да бъде празен.

[StringLength(100)] поставя максимална дължина от 100 символа.

Използваме 100 тук, защото това вече е установеното ограничение в проекта за текстовото поле Email; но **следващата стъпка ще бъде да проверим дали моделът/изискванията имат конкретно правило за FirstName**, преди да добавяме други ограничения.

Сега **не променяй нищо друго**.

Направи промяната и пусни **Build**. Кажи ми резултата от Build-а и продължаваме.

11. Вие казахте:

магическите числа ги изнасяме в файл

12. ChatGPT каза:

Точно така. 👍 Добре, че го хвана.

В проекта сме приели правилото: **магическите числа не ги пишем директно в кода**, а ги изнасяме в StudentConstants.

Затова **не добавяй** [StringLength(100)]. Връщаме се една стъпка и първо проверяваме какво вече имаме.

Стъпка 1 — проверяваме StudentConstants

Отвори:

```
StudentManagement
├── Common
│   └── Constants
│       └── StudentConstants.cs
```

Изпрати ми **целия текущ код** на StudentConstants.cs.

После ще определим дали имаме подходяща константа за FirstName и ще направим само необходимата промяна.

Правило за проекта:

18, 100 и други стойности, които представляват правила/ограничения, **не ги разпръскваме из кода**. Използваме именувани константи.

13. Вие казахте:

```
namespace StudentManagement.Common.Constants; public static
class StudentConstants { public const int MinAge = 18; public const
int MaxAge = 100; }
```

14. ChatGPT каза:

Точно. В момента StudentConstants съдържа само ограниченията за Age:

```
public const int MinAge = 18;
public const int MaxAge = 100;
```

Затова сега правим **само една промяна** — добавяме константа за максималната дължина на името.

Стъпка 1 — добавяме MaxNameLength

Файл

Отвори:

```
Common/Constants/StudentConstants.cs
```

Намери:

```
public const int MinAge = 18;
```

```
public const int MaxAge = 100;
```

и го промени на:

```
public const int MinAge = 18;  
public const int MaxAge = 100;  
public const int MaxNameLength = 100;
```

Коментар

MaxNameLength е именувана константа за максималната дължина на име.

Така вместо да пишем магическото число 100 директно във validation атрибута, по-късно ще използваме:

```
StudentConstants.MaxNameLength
```

Това прави правилото централизирано и по-лесно за промяна.

Сега спираме дотук. Не променяй още Student.cs.

Направи тази промяна и пусни **Build**. Кажи ми резултата.

15. Вие казахте:

```
Build started at 8:48 PM... 1>----- Build started: Project:  
StudentManagement, Configuration: Debug Any CPU -----  
1>StudentManagement ->  
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net  
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0  
failed, 0 up-to-date, 0 skipped ===== Build  
completed at 8:48 PM and took 06.170 seconds =====
```

16. ChatGPT каза:

Отлично. 

Build:

- 1 succeeded
- 0 failed
- 0 errors
- 0 warnings

Константата е добавена успешно.

Стъпка 2 — използваме константата за `FirstName`

Сега вече можем да променим Student.cs.

Файл

Отвори:

```
Models/Student.cs
```

Най-горе, след:

```
using System.ComponentModel.DataAnnotations;
```

добави:

```
using StudentManagement.Common.Constants;
```

Така началото на файла ще стане:

```
using System.ComponentModel.DataAnnotations;
```

```
using StudentManagement.Common.Constants;
```

```
namespace StudentManagement.Models
```

Коментар

Добавяме using, за да може Student.cs да използва:

```
StudentConstants.MaxLength
```

без да изписваме пълния namespace всеки път.

Само тази промяна засега. Не добавяй още [StringLength].

Направи промяната и пусни **Build**. След резултата продължаваме.

17. Вие казахте:

Build started at 8:49 PM... 1>----- Build started: Project:

StudentManagement, Configuration: Debug Any CPU -----

1>StudentManagement ->

D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net

8.0\StudentManagement.dll ===== Build: 1 succeeded, 0

failed, 0 up-to-date, 0 skipped ===== Build

completed at 8:49 PM and took 00.727 seconds =====

18. ChatGPT каза:

Перфектно. 

Build:

- 1 succeeded
- 0 failed
- 0 errors
- 0 warnings

Продължаваме с **една** промяна.

Стъпка 3 — StringLength **за** FirstName

Файл

Отвори:

```
Models/Student.cs
```

Намери:

```
[Required]
```

```
public string FirstName { get; set; } = string.Empty;
```

Замени го с:

```
[Required]
[StringLength(StudentConstants.MaxNameLength)]
public string FirstName { get; set; } = string.Empty;
```

Коментар

Тук вече **не използваме магическото число** 100.

Вместо:

```
[StringLength(100)]
```

използваме:

```
[StringLength(StudentConstants.MaxNameLength)]
```

Така максималната дължина се управлява от едно централизирано място — StudentConstants.cs.

[Required] остава, защото това е отделно validation правило: името е задължително.

Сега само тази промяна. След това пусни Build и ми покажи резултата.

1. Build started at 8:51 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 8:51 PM and took 00.702 seconds =====
2. **ChatGPT каза:**

Отлично. 

Build:

- 1 succeeded
- 0 failed
- 0 errors
- 0 warnings

FirstName вече има две server-side validation правила:

```
[Required]
[StringLength(StudentConstants.MaxNameLength)]
public string FirstName { get; set; } = string.Empty;
```

Но **още не сме приключили с** FirstName. Преди да добавяме HTML ограничения или hint, трябва да проверим как е реализиран CreateStudentViewModel, защото именно той управлява validation-а на Create формата.

Стъпка 4 — проверяваме ViewModel-а

Отвори:

ViewModels/CreateStudentViewModel.cs

и ми изпрати **целия текущ код**.

Няма да променяме нищо още. Първо го виждаме, както сме се разбрали.

1. namespace StudentManagement.ViewModels; using StudentManagement.Common.Constants; using System.ComponentModel.DataAnnotations; public class CreateStudentViewModel { [Required] public string FirstName { get; set; } = string.Empty; [Required] public string LastName { get; set; } = string.Empty; [Required] [EmailAddress] [StringLength(100)] public string Email { get; set; } = string.Empty; [Range(StudentConstants.MinAge, StudentConstants.MaxAge)] public int Age { get; set; } public string Course { get; set; } = string.Empty; }
2. **ChatGPT каза:**

Виждам го. 👍 И тук има нещо важно: CreateStudentViewModel **още няма същото правило за** FirstName, КОЕТО ТОКУ-ЩО ДОБАВИХМЕ

В Student.

В момента е:

```
[Required]
public string FirstName { get; set; } = string.Empty;
```

А в Student вече е:

```
[Required]
[StringLength(StudentConstants.MaxNameLength)]
public string FirstName { get; set; } = string.Empty;
```

Трябва да ги уеднаквим.

Стъпка 5 — FirstName във ViewModel-а

Файл

ViewModels/CreateStudentViewModel.cs

Намери:

```
[Required]
public string FirstName { get; set; } = string.Empty;
```

Замени го с:

```
[Required]
[StringLength(StudentConstants.MaxNameLength)]
public string FirstName { get; set; } = string.Empty;
```

Коментар

Това е важно, защото при **Create** validation-ът се извършва върху CreateStudentViewModel, а не директно върху Student.

Така и двете нива вече използват едно и също правило:

CreateStudentViewModel

↓

MaxNameLength

Student



MaxNameLength

И най-важното — **няма магическо** 100. Използваме:

`StudentConstants.MaxNameLength`

using StudentManagement.Common.Constants; вече го имаш във файла, така че **не добавяме нищо друго**.

Направи само тази промяна → **Build** → изпрати ми резултата.

1. Build started at 8:55 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 8:55 PM and took 00.739 seconds =====

2. ChatGPT каза:

Перфектно. 

Build:

- 1 succeeded
- 0 failed
- 0 errors
- 0 warnings

Вече `FirstName` има еднакво правило и в `Student`, и

в `CreateStudentViewModel`.

Следващото е да преценим **какво трябва да се случва в HTML формата**. Но преди да променяме View-то, както сме се разбрали, първо трябва да видим реалния код.

Стъпка 6 — проверяваме `Create.cshtml`

Файл

Отвори:

`Views/Students/Create.cshtml`

Изпрати ми **целия текущ код** на файла.

Все още не променяй нищо.

След като го видя, ще определим точно:

- има ли смисъл `maxlength` за `FirstName`;
- откъде трябва да идва стойността;

- нужен ли е informational hint;
 - и ще направим само **една** промяна наведнъж.
3. @model StudentManagement.ViewModels.CreateStudentViewModel
@using StudentManagement.Common.Constants <h1>Add Student</h1> <form asp-action="Create" method="post"> <div class="mb-3"> <label asp-for="FirstName" class="form-label"></label> <input asp-for="FirstName" class="form-control" /> </div> <div class="mb-3"> <label asp-for="LastName" class="form-label"></label> <input asp-for="LastName" class="form-control" /> </div> <div class="mb-3"> <label asp-for="Email" class="form-label"></label> <input asp-for="Email" class="form-control" /> </div> <div class="mb-3"> <label asp-for="Course" class="form-label"></label> <input asp-for="Course" class="form-control" /> </div> <div class="mb-3"> <label asp-for="Age" class="form-label"></label> <input asp-for="Age" type="number" min="@StudentConstants.MinAge" max="@StudentConstants.MaxAge" class="form-control" /> <div class="age-hint"> Age → must be between @StudentConstants.MinAge and @StudentConstants.MaxAge. </div> </div> <button type="submit" class="btn btn-primary">Save</button> <a asp-action="Index" class="btn btn-secondary">Cancel </form>

4. ChatGPT каза:

Да, виждам структурата. 👍

При FirstName в момента имаме:

```
<input asp-for="FirstName" class="form-control" />
```

След като добавихме:

```
[StringLength(StudentConstants.MaxNameLength)]
```

Razor/ASP.NET Core вече може да използва това правило, но **за HTML ограничението** искаме браузърът също да знае максималната дължина.

Тук обаче има важна разлика спрямо Age: за FirstName не трябва да пишем ръчно maxlength="100". Искаме стойността да идва от същата константа.

Стъпка 6 — maxlength за FirstName

Файл

Views/Students/Create.cshtml

Намери:

```
<input asp-for="FirstName" class="form-control" />
```

Замени го с:

```
<input asp-for="FirstName"
      maxlength="@StudentConstants.MaxNameLength"
      class="form-control" />
```

Коментар

Така браузърът получава:

maxlength="100"

но в Razor кода **не сме написали магическото** 100.

Стойността идва от:

StudentConstants.MaxNameLength

Получаваме същия принцип, който използвахме при Age:

StudentConstants

↓
100

↓
Razor → HTML

↓
maxlength="100"

[StringLength(...)] остава server-side validation, а maxlength е HTML/UX ограничение.

Само тази промяна. Не добавяме hint за FirstName още.

Направи промяната → **Build** → изпрати ми резултата.

5. Вие казахте:

Build started at 8:58 PM... 1>----- Build started: Project:

StudentManagement, Configuration: Debug Any CPU -----

1>StudentManagement ->

D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 8:58 PM and took 00.771 seconds ===== може
ли да кажа нещо за hint за това поле

6. ChatGPT каза:

Разбира се. И точно сега е добър момент да го обсъдим, преди да
правим следваща промяна. 👍

При Age направихме hint:

Age → must be between 18 and 100.

който се показва **само когато полето е във focus**, без JavaScript.

За FirstName можем да приложим същия принцип, например:

First name → maximum 100 characters.

Но има един важен въпрос: **дали този hint изобщо е необходим.**

Вече имаме:

- Required → показва, че полето е задължително;
- StringLength → validation при прекалено дълго име;
- maxlength → браузърът не позволява въвеждане над лимита.

Затова аз бих казал, че **hint има смисъл само ако искаме потребителят предварително да знае лимита**, както направихме с Age.

Ако искаме последователност между полетата, бих използвал:

```
<div class="first-name-hint">  
    First name → maximum @StudentConstants.MaxNameLength characters.  
</div>
```

и със CSS да го показваме само при focus, точно както age-hint.

Но нека не променяме още кода. Ако решим да го добавим, ще го направим стъпка по стъпка: първо HTML → Build → после CSS → проверка.

1. First name → maximum @StudentConstants.MaxNameLength characters. или да направим друго нещо което в този случай ще е по добре -> First name → остават 45 символа. или нещо подобно
2. **ChatGPT каза:**

Да — **вторият вариант е по-добър като UX**, защото дава информация, която се променя според въведеното име.

Например:

First name → remaining 45 characters.

Това е по-полезно от:

First name → maximum 100 characters.

защото потребителят вижда **колко място му остава**, а не трябва сам да смята.

Но тук има една важна разлика спрямо Age:

За Age

Успяхме да направим всичко с **HTML + CSS**, без JavaScript.

За брояча на FirstName

Ако искаме:

First name → remaining 100 characters.

после:

First name → remaining 94 characters.

и накрая:

First name → remaining 0 characters.

стойността трябва да се променя **при всяко въвеждане**.

Това вече не може да стане само с Razor + CSS. Ще ни трябва малко JavaScript.

И според правилото, което сме си поставили:

Не използваме JavaScript, ако HTML + CSS + Razor могат да свършат работата.

Тук **не могат**, ако искаме динамичен брояч.

Моето предложение

Аз бих избрал именно динамичния вариант:

First name → remaining 100 characters.

Той е по-добър UX и е реално полезен при поле с максимална дължина.

И можем да го направим така, че да се показва **само когато** FirstName **е във focus**, както направихме с Age.

Но ще го направим много внимателно:

HTML → Build → JavaScript → Build → CSS → проверка

Една промяна наведнъж.

Ако си съгласен с динамичния брояч, започваме с **първата малка промяна — само HTML-а**.

3. Вие казахте:

да

4. ChatGPT каза:

Супер. 🙌 Ще направим динамичния брояч, но **стъпка по стъпка**.

Стъпка 7 — добавяме място за брояча

Първо ще направим **само HTML/Razor промяната**. Все още няма JavaScript.

Файл

Отвори:

Views/Students/Create.cshtml

Намери този код:

```
<input asp-for="FirstName"
        maxLength="@StudentConstants.MaxNameLength"
        class="form-control" />
<span asp-validation-for="FirstName" class="text-danger"></span>
```

Замени го с:

```
<input asp-for="FirstName"
      maxlength="@StudentConstants.MaxNameLength"
      class="form-control" />

<div class="first-name-hint">
    First name → remaining @StudentConstants.MaxNameLength
    characters.
</div>

<span asp-validation-for="FirstName" class="text-danger"></span>
```

Коментар

В момента само създаваме **място**, в което по-късно JavaScript ще може да показва оставащите символи.

Използваме:

```
@StudentConstants.MaxNameLength
```

затова началната стойност няма да е магическото 100.

В браузъра ще се получи приблизително:

```
First name → remaining 100 characters.
```

Засега числото **няма да се променя**, защото още не сме добавили JavaScript.

И още не правим CSS — така можем първо да проверим чисто HTML/Razor частта.

Сега

Направи **само тази промяна** → пусни **Build** → изпрати ми резултата.

1. Build started at 9:08 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 9:08 PM and took 00.780 seconds =====

2. ChatGPT каза:

Отлично. 

Build:

- 1 succeeded
- 0 failed
- 0 errors
- 0 warnings

HTML/Razor частта е наред.

Стъпка 8 — подготвяме JavaScript за брояча

Сега ще направим **само една промяна**: ще добавим JavaScript, който изчислява оставащите символи.

Преди това обаче искам да видя как в момента е организиран JavaScript-ът в проекта, за да не създаваме нов файл или структура, която вече съществува.

Файл

Провери папката:

```
wwwroot/js/
```

и ми кажи **какви** `.js` **файлове има вътре**.

Ако има `site.js`, например, изпрати ми неговия текущ код.

Засега нищо не променяй. Първо ще видим съществуващата структура.

1. Вие казахте:

```
// Please see documentation at  
https://learn.microsoft.com/aspnet/core/client-side/bundling-and-  
minification // for details on configuring this project to bundle and  
minify static web assets. // Write your JavaScript code.
```

2. ChatGPT каза:

Да, това е текущият `wwwroot/js/site.js` и виждам, че в момента е практически празен — има само стандартния коментар от ASP.NET Core.

Това е удобно: **нямаме съществуващ JavaScript, който да засегнем**.

Стъпка 8 — добавяме логиката за брояча

Файл

```
wwwroot/js/site.js
```

В момента имаш:

```
// Please see documentation at  
https://learn.microsoft.com/aspnet/core/client-side/bundling-and-  
minification  
// for details on configuring this project to bundle and minify  
static web assets.
```

```
// Write your JavaScript code.
```

Добави след тези коментари:

```
const firstNameInput =  
document.querySelector('input[name="FirstName"]');  
const firstNameHint = document.querySelector('.first-name-hint');
```

```

if (firstNameInput && firstNameHint) {
    const maxLength = firstNameInput.maxLength;

    firstNameInput.addEventListener('input', function () {
        const remainingCharacters = maxLength - this.value.length;

        firstNameHint.textContent =
            `First name → remaining ${remainingCharacters}
characters.`;
    });
}

```

Коментар

Тук правим няколко неща:

```

const firstNameInput =
document.querySelector('input[name="FirstName"]');

```

Намираме `FirstName` полето.

```

const firstNameHint = document.querySelector('.first-name-hint');

```

Намираме нашия `hint`.

```

const maxLength = firstNameInput.maxLength;

```

Много важно: не пишем 100 в JavaScript.

JavaScript взема `maxlength`, който вече идва от:

```

StudentConstants.MaxNameLength

```

```

↓
Create.cshtml
↓
maxlength="100"
↓
JavaScript

```

След това:

```

firstNameInput.addEventListener('input', ...)

```

казва на браузъра:

Всеки път, когато потребителят промени текста, преизчисли оставащите символи.

Например:

```

Празно поле → remaining 100 characters.
"John"       → remaining 96 characters.
"Alexander"  → remaining 91 characters.

```

А тази проверка:

```

if (firstNameInput && firstNameHint)

```

предпазва кода от грешка, ако някой от елементите не съществува на страницата.

Сега само добави този код в `site.js`.

Не променяй `Create.cshtml` и `site.css`.

После направи **Build** и ми изпрати резултата.

3. Вие казахте:

може ли да ми обясниш JS тъй като аз не съм писала до сега само съм го вижда

4. ChatGPT каза:

Разбира се. ❤️ И даже е много добре, че спираш тук. Няма смисъл просто да копираш JavaScript, без да знаеш какво прави. Ще го разгледаме **от самото начало**, с нашия конкретен пример. Няма да добавяме нов код, докато не стане ясно как работи.

1. Какво изобщо е JavaScript?

В нашето приложение имаме три различни технологии, които изпълняват различни роли:

HTML

↓

Какво има на страницата?

CSS

↓

Как изглежда?

JavaScript

↓

Как реагира страницата?

Например нашето поле:

```
<input asp-for="FirstName"
      maxlength="@StudentConstants.MaxNameLength"
      class="form-control" />
```

HTML казва:

Имаме поле `FirstName`, в което могат да се въведат най-много 100 символа.

Но HTML сам по себе си няма да направи:

„Въвели са 37 символа, значи остават 63.“

Тук идва JavaScript.

2. Как JavaScript „вижда“ HTML?

Когато браузърът зареди страницата, HTML-ът се превръща във вътрешна структура, наречена **DOM** (Document Object Model).

Можеш да си го представиш така:

Страница

```
|
├─ h1
├─ form
│   ├── input FirstName
│   ├── div first-name-hint
│   ├── input LastName
│   └─ ...
```

JavaScript може да намира тези елементи и да работи с тях.

3. Първият ред

Имаме:

```
const firstNameInput =  
document.querySelector('input[name="FirstName"]');
```

Нека го разделим.

document

Това е текущата HTML страница.

Можеш да го мислиш като:

„Дай ми достъп до страницата.“

.querySelector(...)

Това означава:

„Намери ми първия HTML елемент, който отговаря на този CSS selector.“

Например:

```
document.querySelector('input[name="FirstName"]');
```

търси:

```
<input name="FirstName">
```

А нашият Razor:

```
<input asp-for="FirstName" ... />
```

генерира HTML с:

```
name="FirstName"
```

Затова JavaScript може да го намери.

4. Какво е `const`?

Това:

```
const firstNameInput = ...
```

означава:

Създай променлива с име `firstNameInput` и запази резултата вътре.

Можеш да го мислиш като кутия:

`firstNameInput`

HTML FirstName input

`const` казва, че самата променлива няма да бъде пренасочвана към друг обект.

5. Следващият ред

```
const firstNameHint = document.querySelector('.first-name-hint');
```

Тук правим същото.

Търсим HTML елемента:

```
<div class="first-name-hint">
```

Точката:

```
.first-name-hint
```

означава:

намери елемент с този CSS class.

Получаваме:

```
firstNameInput
```

↓

```
[ input ]
```

```
firstNameHint
```

↓

```
[ div ]
```

6. А защо имаме if?

Следва:

```
if (firstNameInput && firstNameHint) {
```

Това е условие.

На български:

Ако имаме `firstNameInput` **и** имаме `firstNameHint`, изпълни кода вътре.

&& означава **И**.

Тоест:

```
FirstName input съществува?
```

И

```
FirstName hint съществува?
```

↓

ДА

↓

изпълни кода

Защо ни е това?

Защото `site.js` може да бъде зареден и на други страници.

Например `Index` страницата може изобщо да няма:

```
<input name="FirstName">
```

Тогава:

```
firstNameInput
```

ще бъде `null`.

Проверката предотвратява опит да работим с несъществуващ елемент.

7. Този ред е много интересен

```
const maxLength = firstNameInput.maxLength;
```

Тук казваме:

Вземи максималната дължина от самото HTML поле.

Нашият `Razor`:

```
maxlength="@StudentConstants.MaxNameLength"
```

генерира:

```
maxlength="100"
```

Следователно `JavaScript` може да прочете:

```
firstNameInput.maxLength
```

и получава:

```
100
```


Това е **много важно за архитектурата ни**.

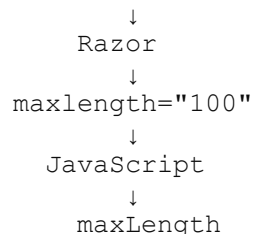
Нямаме:

```
const maxLength = 100;
```

Защото това би било второ място, в което сме записали същото правило.

Вместо това:

```
StudentConstants.MaxNameLength
```



Имаме **един източник на истината**.

8. Най-интересната част — `addEventListener`

Имаме:

```
firstNameInput.addEventListener('input', function () {
```

Това казва:

„Следи това поле и когато се случи събитието `input`, изпълни тази функция.“

Какво е event?

Event = събитие.

Например:

`click`

когато потребителят кликне.

`focus`

когато влезе във поле.

`blur`

когато излезе от поле.

`input`

когато съдържанието на полето се промени.

Ние използваме:

`input`

защото искаме броячът да се променя **докато човек пише**.

9. Какво е `function`?

Имаме:

```
function () {  
    ...  
}
```

Функцията е просто:

парче код, което може да бъде изпълнено.

В нашия случай казваме:

Когато има input event

↓

изпълни тази функция

10. Какво е this?

Вътре имаме:

this.value

Това може да изглежда странно в началото.

В тази функция:

```
firstNameInput.addEventListener('input', function () {
```

this означава **елементът, който е предизвикал събитието**.

Тоест:

this

↓

FirstName input

A:

this.value

означава:

текущият текст в FirstName полето.

Например потребителят е написал:

Alexander

тогава:

this.value

е:

"Alexander"

11. .length

След това:

this.value.length

length означава дължината на текста.

Например:

"John"

има:

4

символа.

A:

"Alexander"

има:

9

символа.

12. Изчислението

Имаме:

```
const remainingCharacters = maxLength - this.value.length;
```

Ако максималната дължина е:

100

и потребителят е написал:

"John"

тогава:

```
maxLength = 100
```

```
this.value.length = 4
```

```
100 - 4 = 96
```

Следователно:

```
remainingCharacters = 96
```

13. Последната част

След това променяме текста на hint-a:

```
firstNameHint.textContent =  
  `First name → remaining ${remainingCharacters} characters.`;
```

textContent означава:

промени текста вътре в HTML елемента.

А това:

```
${remainingCharacters}
```

означава:

постави стойността на променливата тук.

Така:

```
remainingCharacters = 96
```

ВОДИ ДО:

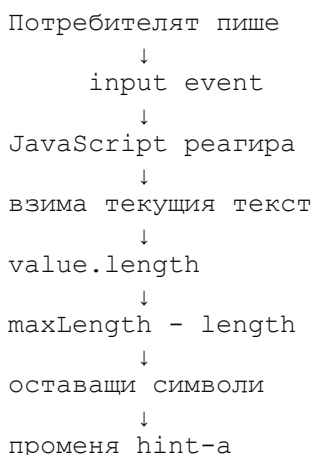
```
First name → remaining 96 characters.
```

След още един символ:

```
First name → remaining 95 characters.
```

Цялата логика като схема

Това е най-важното за запомняне:



Например:

```
FirstName:  
"Maria"
```

```
maxLength = 100  
length = 5
```

```
100 - 5 = 95
```

↓

First name → remaining 95 characters.

И още нещо важно

Не е нужно да запомняш този код наизуст.

В началото е много по-важно да можеш да прочетеш:

```
document.querySelector(...)
```

и да разбереш:

„Търсим HTML елемент.“

Да видиш:

```
addEventListener('input', ...)
```

и да разбереш:

„Чакаме потребителят да промени полето.“

Да видиш:

```
.value.length
```

и да разбереш:

„Взимаме дължината на въведения текст.“

Това вече е **разбиране на JavaScript**, а не просто копиране. 👍

И според нашия начин на работа **не бих продължил със следващата промяна, докато това не ти е ясно.**

1. разбрах го
2. **ChatGPT каза:**

Супер! 👍 Тогава продължаваме по нашия принцип — **една промяна → Build → проверка.**

Стъпка 8 — добавяме JavaScript

Отвори:

```
wwwroot/js/site.js
```

След съществуващите коментари добави **точно този код:**

```
const firstNameInput =  
document.querySelector('input[name="FirstName"]');  
const firstNameHint = document.querySelector('.first-name-hint');  
  
if (firstNameInput && firstNameHint) {  
    const maxLength = firstNameInput.maxLength;  
  
    firstNameInput.addEventListener('input', function () {  
        const remainingCharacters = maxLength - this.value.length;
```

```

        firstNameHint.textContent =
            `First name → remaining ${remainingCharacters}
characters.`;
    });
}

```

Коментар

Това е първата ни реална JavaScript функционалност.

Тя:

- намира `FirstName`;
- намира `.first-name-hint`;
- взема `maxlength` от HTML;
- следи за `input` събитие;
- изчислява оставащите символи;
- обновява текста на `hint`-а.

Не променяй нищо друго.

След това пусни **Build** и ми изпрати резултата.

1. Build started at 9:24 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 9:25 PM and took 01.323 seconds =====
2. **ChatGPT каза:**

Перфектно. 

Build:

- 1 succeeded
- 0 failed
- 0 errors
- 0 warnings

JavaScript кодът се компилира/приема без проблеми.

Сега обаче трябва да проверим **дали работи в браузъра**, защото Build може да е успешен, дори JavaScript логиката да не се изпълнява както очакваме.

Стъпка 9 — проверяваме брояча

Стартирай приложението и отвори **Add Student / Create** страницата.

При `FirstName` провери следното:

5. Когато полето е празно, трябва да виждаш:

First name → remaining 100 characters.

6. Напиши например:

7. John

Трябва да стане:

First name → remaining 96 characters.

8. Добави още един символ и провери дали става:

remaining 95 characters.

9. Изтрий текста и провери дали се връща на:

remaining 100 characters.

Важно

Засега **не правим CSS**, така че `hint`-ът трябва да е видим постоянно.

Това е нарочно — първо проверяваме дали **JavaScript логиката работи**, а след това ще се погрижим визуално да се показва само когато `FirstName` е във `focus`, както направихме при `Age`.

Провери го в браузъра и ми кажи **какво се случва**.

1. в момента работи правилно

2. **ChatGPT каза:**

Супер! 🎉 Това е точно резултатът, който искахме.

В момента имаме:

`StudentConstants`

↓

`MaxNameLength`

↓

`Create.cshtml`

↓

`maxlength`

↓

`JavaScript`

↓

динамичен брояч

И броячът реално работи. ✓

Сега можем да направим следващата малка стъпка: **да го направим като `Age hint`-а — да се показва само когато `FirstName` полето е във `focus`.**

Стъпка 10 — CSS за `FirstName hint`

Файл

Отвори:

`wwwroot/css/site.css`

Намери съществуващия код за Age:

```
.age-hint {  
    display: none;  
}  
  
input[name="Age"]:focus ~ .age-hint {  
    display: block;  
}
```

Веднага след него добави:

```
.first-name-hint {  
    display: none;  
}  
  
input[name="FirstName"]:focus ~ .first-name-hint {  
    display: block;  
}
```

Коментар

Правим абсолютно същата логика като при Age.

Когато FirstName **не е във focus**:

hint → hidden

Когато кликнеш в FirstName:

FirstName → focus

↓

.first-name-hint

↓

display: block

И понеже JavaScript вече променя текста на hint-a, ще получим:

First name → remaining 100 characters.

След като напишеш Maria:

First name → remaining 95 characters.

Важно

Не променяй JavaScript-а.

Не променяй Create.cshtml.

Само добави тези 6 реда в site.css.

След това:

Build → стартирай приложението → провери FirstName.

Този път искам да провериш две неща:

- hint-ът скрит ли е, когато полето не е избрано;
- появява ли се и брой ли правилно, когато пишеш.

3. Вие казахте:

може ли да му сложим и минимална дължина

4. ChatGPT каза:

Да, можем — и според мен е добре да го направим, **ако това е реално изискване за** FirstName. 👍

Но тук искам да спазим нашето правило: **не измисляме validation ограниченията.**

В момента знаем само:

```
FirstName
├─ Required
└─ MaxNameLength = 100
```

За минимална дължина трябва да знаем какво изисква проектът — например дали минимумът е 2, 3 или друга стойност.

И понеже не искаме да създаваме магическо число, правилният ред ще бъде:

```
StudentConstants
    ↓
MinNameLength
    ↓
Student
    ↓
CreateStudentViewModel
    ↓
HTML
```

Ако например изискването е минимум **2 символа**, ще имаме:

```
public const int MinNameLength = 2;
```

и после:

```
[StringLength(
    StudentConstants.MaxNameLength,
    MinimumLength = StudentConstants.MinNameLength)]
```

Но **не добавяй това още.**

Кажете ми какво е зададеното изискване за **минимална дължина на FirstName** (ако го имаш в условието), и ще го направим стъпка по стъпка.

1. по наша преценка
2. **ChatGPT каза:**

Да. Ако ограничението е **по наша преценка**, бих предложил **2 символа минимум.**

Причината е практична: а може технически да е име/инициал, но за поле **First name** минимум 2 символа е разумно UX/validation правило, без да е прекалено рестриктивно.

И ще го направим както досега — **без магическо число.**

Стъпка 11 — MinNameLength

Първо променяме **само** StudentConstants.cs.

Отвори:

```
Common/Constants/StudentConstants.cs
```

В момента имаш:

```
namespace StudentManagement.Common.Constants;
```

```
public static class StudentConstants
{
```



```
    public const int MinAge = 18;
    public const int MaxAge = 100;
    public const int MaxNameLength = 100;
}
```

Добави:

```
public const int MinNameLength = 2;
```

Така целият файл ще бъде:

```
namespace StudentManagement.Common.Constants;
```

```
public static class StudentConstants
{
    public const int MinAge = 18;
    public const int MaxAge = 100;
    public const int MaxNameLength = 100;
    public const int MinNameLength = 2;
}
```

Коментар

Сега имаме централизирани правила:

MinNameLength → 2

MaxNameLength → 100

И по-нататък няма да пишем:

```
MinimumLength = 2
```

а:

```
MinimumLength = StudentConstants.MinNameLength
```

Само добави константата. После Build и ми покажи резултата.

3. Вие казахте:

Има ли собствено име с 2 символа

4. ChatGPT каза:

Да, има. 😊 Но са сравнително редки.

Примери:

- **Bo** — среща се като собствено име.
- **Ed** — собствено име/кратка форма на Edward.
- **Jo** — използва се като собствено име.
- **Li** — среща се като собствено име, особено в китайски контекст.

Затова **минимум 2 символа е напълно разумен**, ако целта ни е да отхвърлим очевидно невалидни стойности като а, без да ограничаваме реални имена.

Само бих направил още едно уточнение: MinimumLength = 2 **не означава**, че името задължително трябва да съдържа само букви. Това е отделно правило. Ако по-късно решим, че John123 например не

трябва да бъде приемливо име, ще трябва да обсъдим отделна validation логика.

За момента бих останал с:

```
public const int MinNameLength = 2;
public const int MaxNameLength = 100;
```

и продължаваме стъпка по стъпка.

1. може ли имената да са MinFirstNameLength и MaxFirstNameLength, поради факта че имаме и FirstName и LastName
2. **ChatGPT казва:**

Да — **това е по-доброто именуване.** 👍

Понеже имаме отделни полета FirstName И LastName, MinNameLength е малко неясно. Ако утре решим, че LastName има различни ограничения, ще стане объркващо.

По-ясно е:

```
public const int MinFirstNameLength = 2;
public const int MaxFirstNameLength = 100;
```

Така името на константата само по себе си казва **за кое поле и за какво правило** се отнася.

Стъпка 11 — преименуваме константата

Тъй като вече добавихме MaxNameLength, не добавяй нови константи до нея. Просто промени:

```
public const int MaxNameLength = 100;
public const int MinNameLength = 2;
```

на:

```
public const int MinFirstNameLength = 2;
public const int MaxFirstNameLength = 100;
```

Коментар

Получаваме:

FirstName

```
├─ MinFirstNameLength = 2
└─ MaxFirstNameLength = 100
```

Това ни оставя добра основа и за LastName:

FirstName

```
├─ MinFirstNameLength
└─ MaxFirstNameLength
```

LastName

```
├─ MinLastNameLength
└─ MaxLastNameLength
```

Ако някой ден FirstName и LastName имат различни ограничения, кодът ще е напълно ясен.

Важно: понеже MaxNameLength вече се използва

В Student.cs И CreateStudentViewModel.cs, след преименуването тези две

места временно ще дадат грешка, докато не ги обновим. Това е очаквано.

Затова засега направи **само промяната в** StudentConstants.cs. Не поправяй останалите файлове още.

После ми кажи какво показва Visual Studio.

3. Вие казахте:

ние има и двете и firstName и lastName

4. ChatGPT каза:

Да, точно това е причината да не използваме общото NameLength.



Имаме **две отделни свойства**:

FirstName

LastName

и е по-добре всяко да има ясно именувани ограничения.

Аз бих предложил структурата:

```
public const int MinFirstNameLength = 2;  
public const int MaxFirstNameLength = 100;
```

```
public const int MinLastNameLength = 2;  
public const int MaxLastNameLength = 100;
```

Това е по-ясно от:

MinNameLength

MaxNameLength

защото вторият вариант не казва **кое име** имаме предвид.

Но нека не бързаме

В момента работим върху FirstName, затова **не добавяме още ограничения за** LastName.

За FirstName първо ще направим:

MinFirstNameLength

MaxFirstNameLength

После ще приложим тези константи последователно в:

Student

↓

CreateStudentViewModel

↓

Create.cshtml

↓

JavaScript

А когато стигнем до LastName, ще вземем отделно решение за неговите ограничения.

Така няма да смесваме двете полета и ще запазим проекта чист.



Следващото действие: В StudentConstants.cs ЗАМЕНИ:

```
public const int MaxNameLength = 100;  
public const int MinNameLength = 2;
```

С:

```
public const int MinFirstNameLength = 2;  
public const int MaxFirstNameLength = 100;
```

После **Build**. Грешките, които ще се появят заради старото име MaxNameLength, са очаквани — следващата ни стъпка ще бъде да ги поправим една по една.

5. Вие казахте:

```
namespace StudentManagement.Common.Constants; public static  
class StudentConstants { public const int MinAge = 18; public const  
int MaxAge = 100; public const int MinFirstNameLength = 2; public  
const int MaxFirstNameLength = 100; }
```

6. ChatGPT каза:

Да, така е добре. 📌 Имената вече са конкретно свързани
с FirstName.

Имаме:

```
public const int MinFirstNameLength = 2;  
public const int MaxFirstNameLength = 100;
```

Това е чисто и разбираемо.

Стъпка 12 — Student.cs

Сега трябва да заменим старото име на константата, защото

в Student.cs все още използваме:

```
StudentConstants.MaxNameLength
```

Файл

Отвори:

Models/Student.cs

Намери:

```
[StringLength(StudentConstants.MaxNameLength)]
```

и го замени с:

```
[StringLength(StudentConstants.MaxFirstNameLength)]
```

Така целият участък ще бъде:

```
[Required]  
[StringLength(StudentConstants.MaxFirstNameLength)]  
public string FirstName { get; set; } = string.Empty;
```

Коментар

Тук просто казваме по-точно:

FirstName има максимална дължина, определена от MaxFirstNameLength.

Все още **не добавяме минималната дължина** към validation-a.

Правим само поправката на името на константата.

След тази промяна **не пипай други файлове**.

Пусни **Build** и ми изпрати резултата.

1. 'StudentConstants' does not contain a definition for 'MaxNameLength'
2. **ChatGPT каза:**

Да — това е **очакваната грешка** след преименуването. 👍

Ние преименувахме:

MaxNameLength

на:

MaxFirstNameLength

НО CreateStudentViewModel.cs все още използва старото име.

Няма проблем — оправяме **само това**.

Стъпка 13 — поправяме CreateStudentViewModel

Файл

Отвори:

ViewModels/CreateStudentViewModel.cs

Намери:

```
[StringLength(StudentConstants.MaxNameLength)]  
public string FirstName { get; set; } = string.Empty;
```

Замени го с:

```
[StringLength(StudentConstants.MaxFirstNameLength)]  
public string FirstName { get; set; } = string.Empty;
```

Коментар

Това е същата промяна, която направихме в Student.cs.

Вече и двете места използват новото, по-точно име:

Student

↓

MaxFirstNameLength

CreateStudentViewModel

↓

MaxFirstNameLength

След това пусни **Build**.

Очакваме:

Build: 1 succeeded

Errors: 0

Warnings: 0

Изпрати ми резултата и продължаваме с **минималната дължина**.

3. **Вие казахте:**

4 грешки

4. ChatGPT каза:

Да, спокойно — **нека не гадаем кои са 4-те грешки**. Това е точно моментът да ги видим.

След преименуването е напълно възможно да има още места, които използват старото:

```
StudentConstants.MaxNameLength
```

Сега не променяй нищо повече.

Моля, копирай ми **четирите грешки от Error List** — най-добре целия текст на 4-те реда.

Например:

```
CS0117 'StudentConstants' does not contain a definition for  
'MaxNameLength'  
...
```

Искам да видя **точно файловете и редовете**, за да ги поправим един по един, без да пипаме нещо излишно.

След това ще продължим оттам.

1. 'StudentConstants' does not contain a definition for
'MaxNameLength' 'StudentConstants' does not contain a definition
for 'MaxNameLength' 'StudentConstants' does not contain a
definition for 'MaxNameLength'

2. ChatGPT каза:

Да — вече е ясно какво се е случило. 👍

Трите грешки са от **три останали места**, които още търсят старото име:

```
StudentConstants.MaxNameLength
```

Няма да променяме на сляпо. Ще ги намерим.

Стъпка 14 — намираме старото име

В Visual Studio натисни:

Ctrl + Shift + F

Това отваря **Find in Files**.

В полето за търсене напиши точно:

```
StudentConstants.MaxNameLength
```

Избери търсене в **Entire Solution**.

Какво правим сега?

Само търсим. **Не променяй още нищо.**

Трябва да видим трите места, които използват старото име.

Изпрати ми резултата от търсенето — **файл + ред**, например:

```
Create.cshtml    → ...  
Student.cs       → ...  
site.js          → ...
```

После ще ги поправим **едно по едно** и ще направим Build.

1. Create -> 16
2. ChatGPT каза:

Точно. 👍 Първото място е в Create.cshtml, на **ред 16**.

Стъпка 14.1 — поправяме Create.cshtml

Файл

Views/Students/Create.cshtml

Отиди на **ред 16**.

Вероятно имаш:

```
<input asp-for="FirstName"
        maxLength="@StudentConstants.MaxNameLength"
        class="form-control" />
```

Замени само:

StudentConstants.MaxNameLength

с:

StudentConstants.MaxFirstNameLength

Така кодът ще бъде:

```
<input asp-for="FirstName"
        maxLength="@StudentConstants.MaxFirstNameLength"
        class="form-control" />
```

Коментар

Тук просто актуализираме използването на константата след преименуването ѝ.

Преди:

MaxNameLength

Сега:

MaxFirstNameLength

Това е по-точно, защото ограничението се отнася конкретно до FirstName.

Не прави Build още. Имаме още две места със старото име.

След промяната отново направи:

Ctrl + Shift + F → StudentConstants.MaxNameLength

и ми кажи **какви са следващите резултати**.

3. Вие казахте:

create -> 12

4. ChatGPT каза:

Точно. 👍 Значи имаме още едно място в Create.cshtml, на **ред 12**.

Стъпка 14.2 — поправяме Create.cshtml

Отиди на **ред 12**.

Най-вероятно там е hint-ът:

```
<div class="first-name-hint">  
    First name → remaining @StudentConstants.MaxNameLength  
characters.  
</div>
```

Замени само:

```
StudentConstants.MaxNameLength
```

с:

```
StudentConstants.MaxFirstNameLength
```

Трябва да стане:

```
<div class="first-name-hint">  
    First name → remaining @StudentConstants.MaxFirstNameLength  
characters.  
</div>
```

Коментар

Тук също използваме вече преименуваната константа.

Това е особено важно за нашия динамичен брояч: **началната стойност на hint-а трябва да идва от същата константа**, от която идва и maxlength.

След тази промяна **не прави Build още**.

Направи отново:

Ctrl + Shift + F

и потърси:

```
StudentConstants.MaxNameLength
```

Трябва да остане **още едно място**. Кажете ми кое е то.

5. Вие казахте:

```
students->12
```

6. ChatGPT каза:

Да. 👍 Последното място е в Students — **ред 12**.

Тук вече трябва да внимаваме да не предполагаме какъв е кодът.

Затова:

Стъпка 14.3 — виждаме ред 12

Отвори файла, който Visual Studio показва като:

```
Students
```

и отиди на **ред 12**.

Не променяй още нищо.

Копирай ми ред 12 (може и 2–3 реда около него), например:

```
10 ...  
11 ...  
12 ...  
13 ...
```


Ще ти кажа точно какво да замениш. Така няма да гадаем къде се използва MaxNameLength. 👍

1. Вие казахте:

```
[Required] [StringLength(StudentConstants.MaxNameLength)] public  
string FirstName { get; set; } = string.Empty;
```

2. ChatGPT каза:

Да, това е третото и последно място. 👍

Стъпка 14.3 — поправяме Students

Намери:

```
[Required]  
[StringLength(StudentConstants.MaxNameLength)]  
public string FirstName { get; set; } = string.Empty;
```

Замени **само**:

```
StudentConstants.MaxNameLength
```

с:

```
StudentConstants.MaxFirstNameLength
```

Трябва да стане:

```
[Required]  
[StringLength(StudentConstants.MaxFirstNameLength)]  
public string FirstName { get; set; } = string.Empty;
```

Коментар

Това е третото място, което използваше старото име.

Сега всички употреби на:

```
StudentConstants.MaxNameLength
```

трябва да са заменени с:

```
StudentConstants.MaxFirstNameLength
```

Сега

Направи отново:

Ctrl + Shift + F

и потърси:

```
StudentConstants.MaxNameLength
```

Ако търсенето покаже **0 резултата**, тогава вече пускаме **Build**.

Засега не променяме нищо друго.

3. Вие казахте:

Build started at 9:53 PM... 1>----- Build started: Project:

StudentManagement, Configuration: Debug Any CPU -----

1>StudentManagement ->

D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net

8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 9:53 PM and took 01.185 seconds =====

4. ChatGPT каза:

Отлично. ☒ Вече сме изчистили преименуването.

Build:

- 1 succeeded
- 0 failed
- 0 errors
- 0 warnings

И сега имаме правилно именувани константи:

```
public const int MinFirstNameLength = 2;  
public const int MaxFirstNameLength = 100;
```

Следващата логична стъпка е да **използваме** `MinFirstNameLength` **в validation-a**.

Стъпка 15 — минимална дължина в Student.cs

Файл

Models/Student.cs

В момента имаш:

```
[Required]  
[StringLength(StudentConstants.MaxFirstNameLength)]  
public string FirstName { get; set; } = string.Empty;
```

Замени го с:

```
[Required]  
[StringLength(  
    StudentConstants.MaxFirstNameLength,  
    MinimumLength = StudentConstants.MinFirstNameLength)]  
public string FirstName { get; set; } = string.Empty;
```

Коментар

Тук вече използваме **двете константи**:

```
MinFirstNameLength → 2  
MaxFirstNameLength → 100
```

`StringLength` ще казва:

`FirstName` трябва да бъде между 2 и 100 символа.

`[Required]` остава отделно, защото неговата задача е да казва: полето не може да бъде празно.

Така имаме:

```
FirstName  
├── Required  
└── StringLength
```

```
└─ minimum = 2
└─ maximum = 100
```

Само тази промяна засега.

Пусни **Build** и ми изпрати резултата.

1. Build started at 9:59 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 9:59 PM and took 00.770 seconds =====
2. **ChatGPT каза:**

Отлично — Build-ът е чист. 

- **1 succeeded**
- **0 failed**
- **0 errors**
- **0 warnings**

Вече Student.cs има:

```
[Required]
[StringLength(
    StudentConstants.MaxFirstNameLength,
    MinimumLength = StudentConstants.MinFirstNameLength)]
public string FirstName { get; set; } = string.Empty;
```

Така FirstName има:

- минимум **2** символа;
- максимум **100** символа;
- задължително поле.

Следваща стъпка

Сега трябва да приложим **същото правило** в CreateStudentViewModel, защото Create формата валидира ViewModel-a.

Отвори:

ViewModels/CreateStudentViewModel.cs

Намери:

```
[Required]
[StringLength(StudentConstants.MaxFirstNameLength)]
public string FirstName { get; set; } = string.Empty;
```

и го замени с:

```
[Required]
[StringLength(
    StudentConstants.MaxFirstNameLength,
```

```
        MinimumLength = StudentConstants.MinFirstNameLength)]  
public string FirstName { get; set; } = string.Empty;
```

Коментар

Това е същото правило, което току-що добавихме в Student.

Искаме:

CreateStudentViewModel

↓
2 - 100 СИМВОЛА

↓
Student

↓
2 - 100 СИМВОЛА

Така Create формата и моделът няма да имат различни правила.

Само тази промяна. След нея → Build → изпрати ми резултата.

1. Build started at 10:01 PM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 10:01 PM and took 00.761 seconds =====

2. ChatGPT каза:

Отлично — отново чист Build. 

- **1 succeeded**
- **0 failed**
- **0 errors**
- **0 warnings**

Сега FirstName има еднаква server-side validation логика и в двете
места:

Student

↓
2 - 100 СИМВОЛА

CreateStudentViewModel

↓
2 - 100 СИМВОЛА

Стъпка 16 — HTML minlength

Сега можем да дадем същото ограничение и на брауъра.

Файл

Отвори:

Views/Students/Create.cshtml

Намери:

```
<input asp-for="FirstName"
```

```
maxlength="@StudentConstants.MaxFirstNameLength"
class="form-control" />
```

Замени го с:

```
<input asp-for="FirstName"
    minlength="@StudentConstants.MinFirstNameLength"
    maxlength="@StudentConstants.MaxFirstNameLength"
    class="form-control" />
```

Коментар

Добавяме:

```
minlength="@StudentConstants.MinFirstNameLength"
```

Това ще генерира:

```
minlength="2"
```

Така браузърът също знае, че FirstName трябва да има поне 2 символа.

Имаме вече три нива:

StudentConstants

↓
2 - 100
↓

Server Browser/HTML

→ [StringLength]
→ minlength / maxlength

И отново — **никакви магически числа** в Create.cshtml.

Само тази промяна → **Build** → изпрати резултата.

1. Build started at 10:03 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 10:03 PM and took 00.792 seconds =====
2. **ChatGPT каза:**

Отлично — чист Build отново. 

- **1 succeeded**
- **0 failed**
- **0 errors**
- **0 warnings**

Вече FirstName има:

Required

↓
не може да е празно

StringLength

↓
2 - 100 символа

HTML

↓
minlength="2"
maxlength="100"

И всички стойности идват от StudentConstants. Това е точно архитектурата, която искаме.

Следващата стъпка — проверка в браузъра

Преди да добавяме още код, нека проверим какво реално генерира Razor.

Стартирай приложението и отвори **Add Student**.

В FirstName:

5. Остави полето празно и натисни **Save**.

6. Въведи **1 символ**, например:

7. A

и натисни **Save**.

8. Въведи **2 символа**, например:

9. Al

и провери дали вече е допустимо.

10. Провери и брояча:

11. First name → remaining 100 characters.

след A1 трябва да стане:

First name → remaining 98 characters.

Защо правим тази проверка?

Искаме да видим как се държат заедно:

HTML validation
+
ASP.NET validation
+
JavaScript counter

Особено интересно е как браузърът реагира на **1 символ** след добавянето на minlength.

Не правим нова промяна още.

Провери тези неща и ми кажи **какво точно се случва при A и при празно поле**.

1. Рорир от browser -> моля, удължете този текст до поне 2 знака.(понастоящем използвате 1знак)

2. ChatGPT каза:

Да — **точно това очаквахме.** ✓

Browser popup-ът:

„Моля, удължете този текст до поне 2 знака. (понастоящем използвате 1 знак)“

показва, че minlength="2" работи.

Това е хубав пример как различните нива си вършат работата:

Въвеждаме: A
↓
HTML minlength="2"
↓
Browser validation
↓
Popup

А при **2 символа** например AI браузърът вече няма да блокира заради минималната дължина.

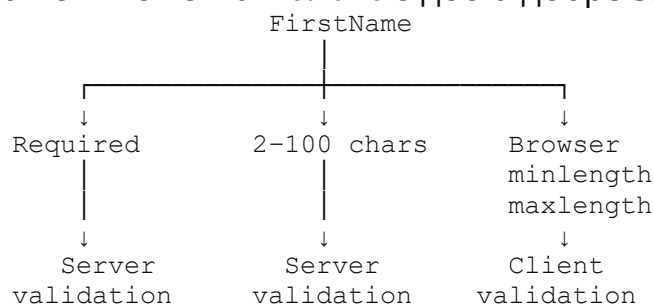
Важен детайл

Това е **browser validation**, не съобщението от:

[StringLength(..., MinimumLength = ...)]

ASP.NET Core validation остава отделна server-side защита.

Така че в момента FirstName е доста добре защитен:



Следващото, което трябва да решим, е **дали искаме динамичният hint да показва и минималното изискване**, или да остане само броячът:

First name → remaining 99 characters.

Аз лично бих запазил брояча, защото той е по-полезен като UX, а minlength вече се грижи за минималното ограничение.

Но преди следващата промяна можем да обсъдим и **как да изглежда hint-ът визуално**, за да е последователен с Age.

3. Вие казахте:

ОК

4. ChatGPT каза:

Супер. Тогава **не променяме hint-а засега**. Оставяме:

`First name → remaining 100 characters.`

и JavaScript го обновява според въведеното.

Следваща стъпка

Преди да преминем към `LastName`, бих направил още една важна проверка на `FirstName`:

- 0 символа → Required
- 1 символ → `minlength="2"`
- 2 символа → допустимо
- 100 символа → допустимо
- 101 символа → `maxlength="100"`

Това ще ни потвърди, че **цялата логика за `FirstName` работи както трябва**, преди да започнем `LastName`.

Ако искаш, можем да направим тази проверка **една по една**, както досега.

5. Вие казахте:

може ли да добавим по * до името на label и някъде долу да кажем че *- задължително поле което трябва да има стойност

6. ChatGPT каза:

Да, и според мен това е **по-добър UX**. Ще стане ясно кои полета са задължителни, без да разчитаме само на validation съобщението.

Ще го направим **без да променяме validation логиката**.

Стъпка 17 — * на задължителните полета

Първо ще направим **само `FirstName`**, за да не променяме няколко неща наведнъж.

Файл

`Views/Students/Create.cshtml`

В момента имаш:

```
<label asp-for="FirstName" class="form-label"></label>
```

Замени го с:

```
<label asp-for="FirstName" class="form-label">
  First name *
</label>
```

Коментар

Тук просто заменяме автоматично генерирания текст на label-a с наш собствен текст:

First name *

* означава:

това поле е задължително.

Но: самата звезда не прави полето задължително. Това вече се гарантира от:

[Required]

Тоест:

*	→ визуална подсказка за потребителя
[Required]	→ реална validation логика

Стъпка 18 — обяснението на *

След това ще добавим **едно общо обяснение в края на формата**, например:

```
<div class="required-fields-note">  
    * Required field  
</div>
```

Но не го добавяй още.

Първо направи само промяната на FirstName label-a:

```
<label asp-for="FirstName" class="form-label">  
    First name *  
</label>
```

После ми покажи как изглежда страницата или просто ми кажи дали First name * се вижда правилно. След това ще добавим общото обяснение долу.